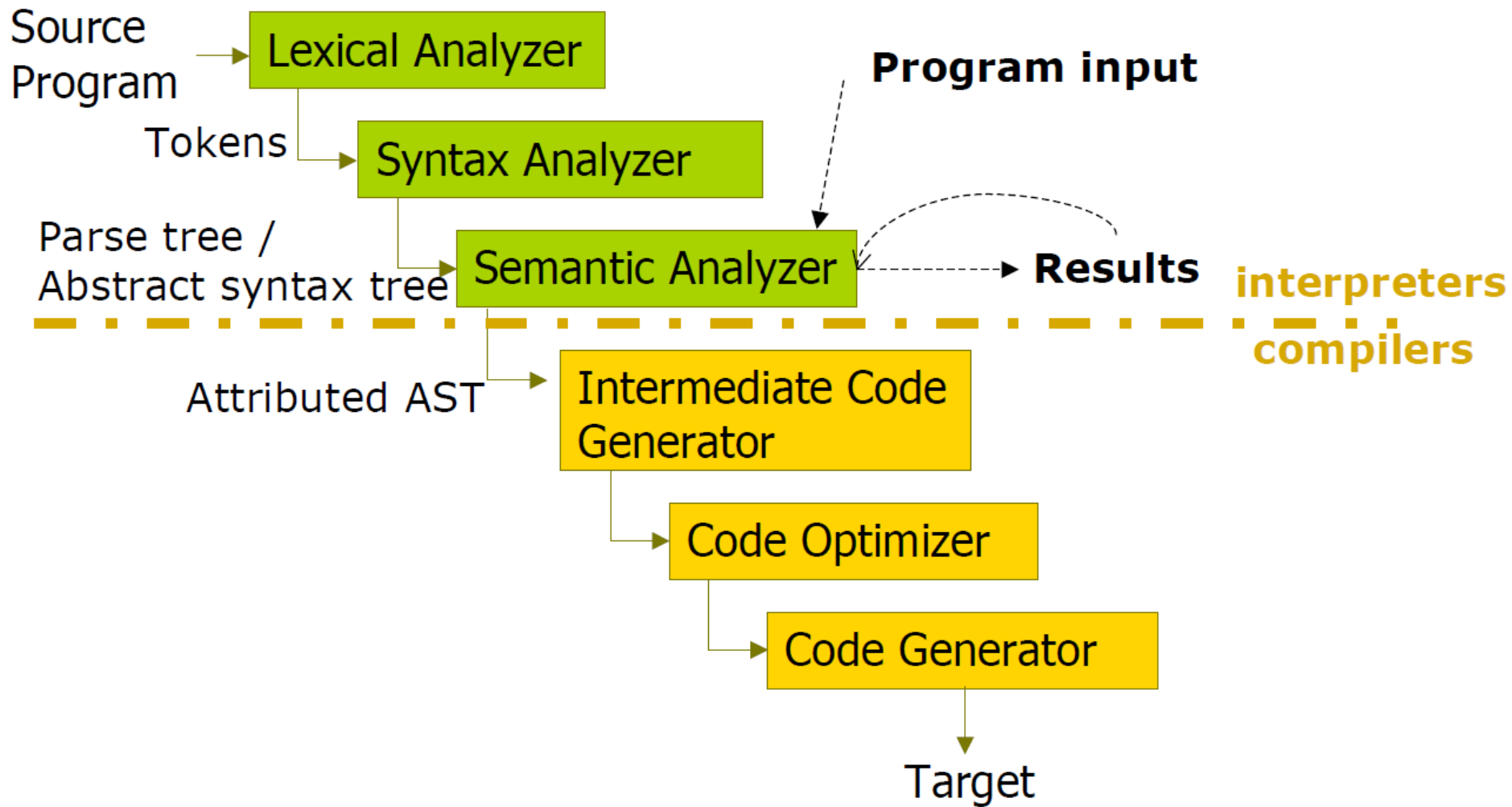


第七章 运行时环境

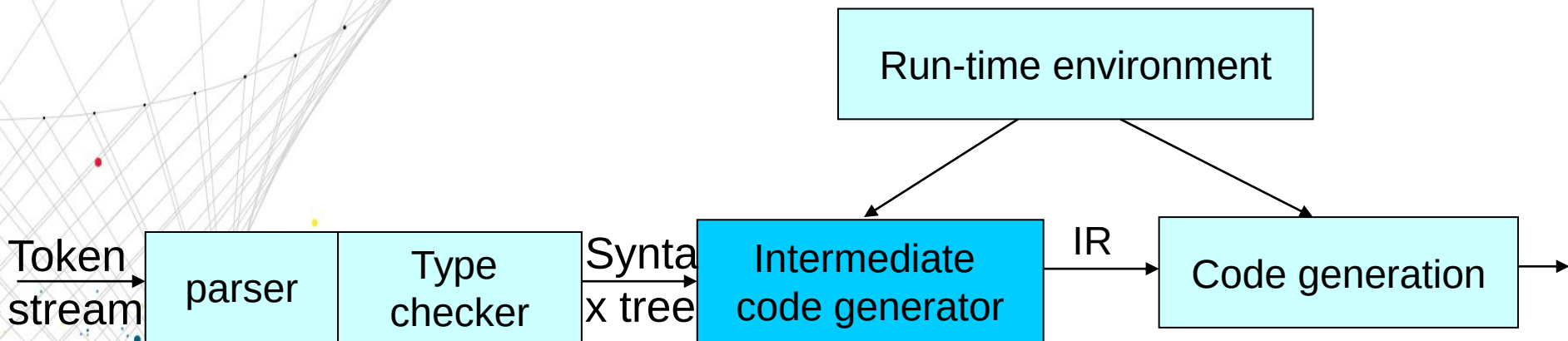
张敏

软件工程学院 华东师范大学

问题探索



问题探索



◆ 编译器后端需要什么样得环境？

存 储

目标程序运行时的活动

假定程序由若干个过程组成

- 过程（**procedure**）定义
- 一个过程的**活动(Activation)**：该过程的一次执行
- 过程**P**一个**活动的生存期(Lifetime)**：从执行该过程体第一步操作到最后一步操作之间的操作序，包括执行**P**时调用其它过程花费的时间
- 过程可以是**递归**的

```
(1) program sort(input, output)
(2)   var a: array[0..10] of integer;
(3)   procedure readarray;
(4)     var i: integer;
(5)   begin
(6)     for i:=1 to 9 do read(a[i])
(7)   end;
(8)   function partition(y, z:integer):integer;
(9)     var i:integer;
(10)   begin .....
(11)   end;
(12)   procedure quicksort(m, n:integer);
(13)     var i:integer;
(14)   begin
(15)     if (n>m) then begin
(16)       i:=partition(m, n );
(17)       quicksort(m, i-1 );
(18)       quicksort(i+1, n )
(19)     end;
(20)   end;
(21) begin
(22)   a[0]:=-9999; a[10]:=9999;
(23)   readarray;
(24)   quicksort(1, 9 )
(25) end.
```

program sort

procedure readarray

function partition

procedure quicksort

目标程序运行时的活动

➤ 过程定义:

```
procedure add(x,y:integer; var z:integer)
begin
    z:=x+y;
end;
```

➤ 过程调用

```
add(a,b,c);
```

过程是模块程序设计的主要手段，是节省程序代码和扩充语言的主要途径

活动树 (Activation Tree)

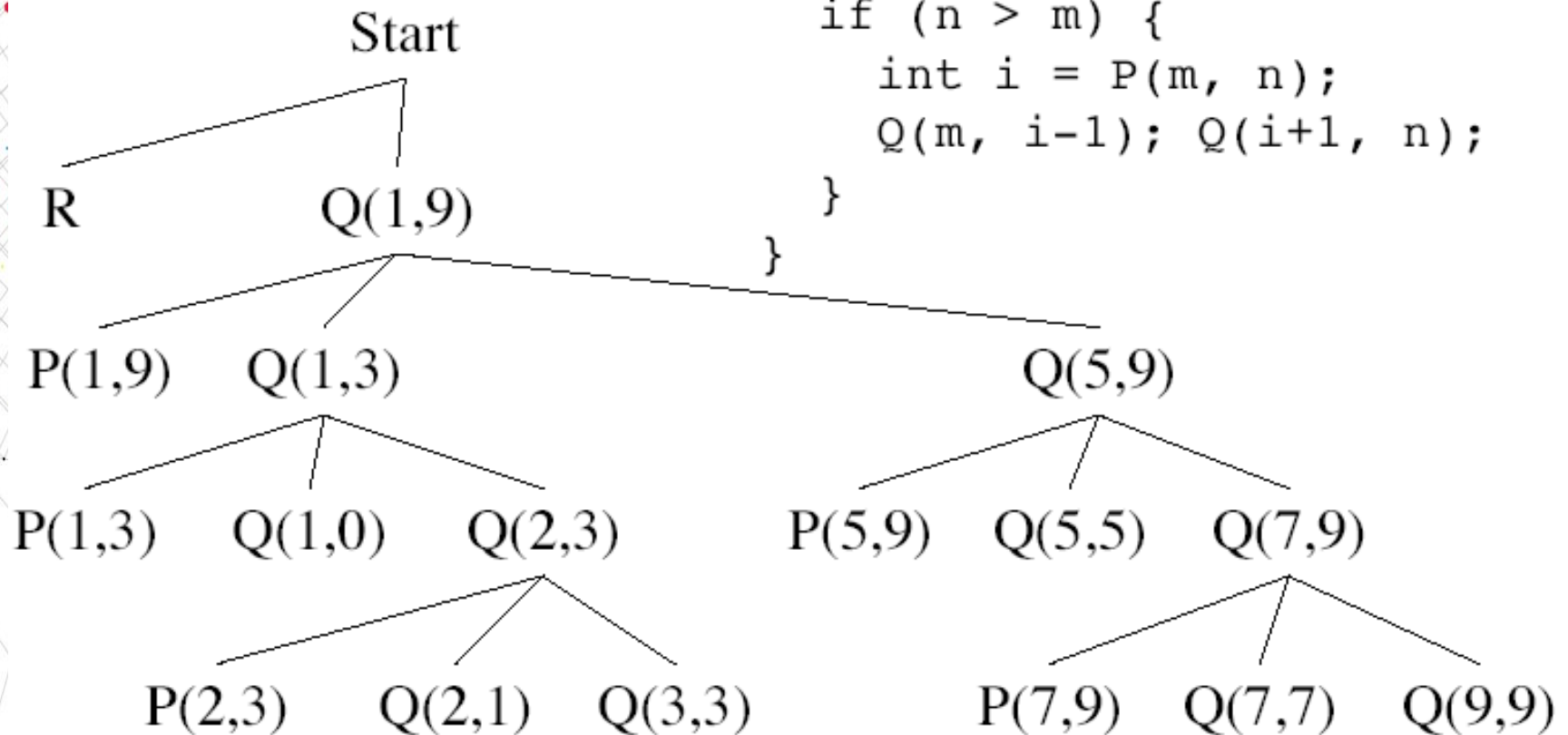
活动树中：

- 每一个节点代表一个活动
- 根节点代表主程序的活动
- 当且仅当控制流从活动a到活动b时，节点a就是节点b的父节点
- 当且仅当a的生存周期先于b的生存周期发生时，节点a在节点b的左边。

我们用一棵树来描绘控制进入和离开活动的方式

活动树 (Activation Tree)

```
int Q(int m, int n) {  
    if (n > m) {  
        int i = P(m, n);  
        Q(m, i-1); Q(i+1, n);  
    }  
}
```



活动树 (Activation Tree)

```
program main;  
  procedure s;  
    begin ... end;  
  procedure p;  
    procedure q;  
      begin ... end;  
    begin q; s; end;  
  begin p; s; end;
```

enter main
 enter p
 enter q
 exit q
 enter s
 exit s
exit p
 enter s
 exit s
exit main

A Nested Structure

参数传递

➤ 传地址

把实在参数的地址传递给相应的形式参数

➤ 得结果

传地址的一种变形

➤ 传值

把实在参数的值传递给相应的形式参数

➤ 传名

过程调用的作用相当于把被调用段的过程体抄到调用出现的地方，但把其中任一出现的形式参数都替换成相应的实参。

参数传递

例:

...

```
procedure P(w,x,y,z);
```

```
begin
```

```
  y := y*w;
```

```
  z := z+x;
```

```
end
```

```
begin
```

```
  a := 5;
```

```
  b := 3;
```

```
  P(a+b,a-b,a,a);
```

```
  write(a);
```

```
end
```

传值: 5

传地址: 42

得结果: 7

传名: 77

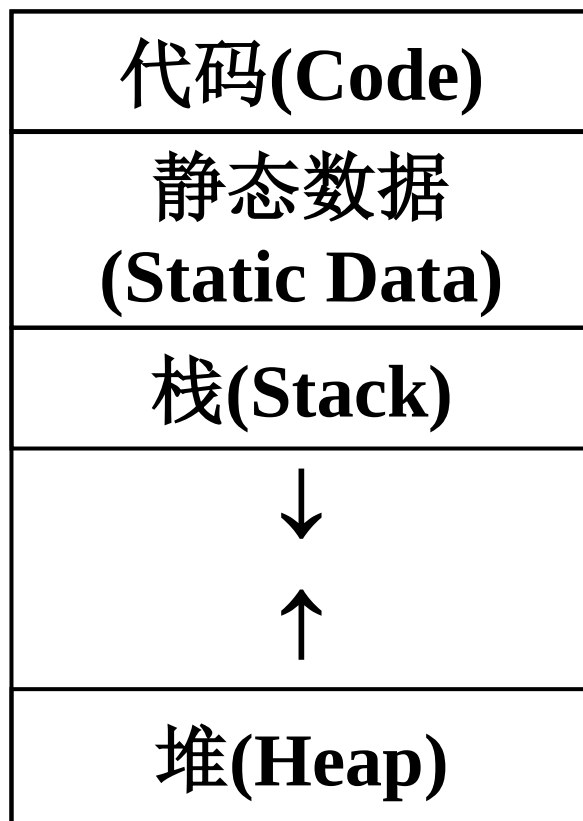
编译程序为了组织存储空间，必须考虑下面几个问题：

- 过程是否允许递归？
- 当控制从一个过程的活动返回时，对局部名称的值如何处理？
- 过程是否允许引用非局部名称？
- 过程调用时如何传递参数；过程是否可以做为参数被传递和做为结果被返回？
- 存储空间可否在程序控制下进行动态分配？
- 存储空间是否必须显式地释放？

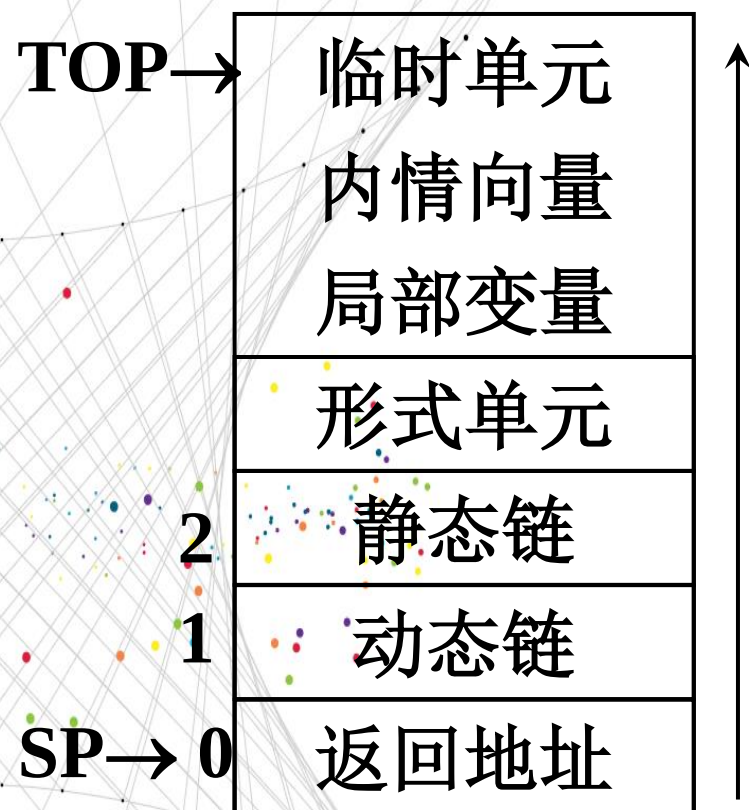
运行时存储器的划分

一个目标程序运行所需的存储空间包括:

- 存放目标代码的空间
- 存放数据项目的空间
- 存放程序运行的控制或连接数据所需单元(控制栈)



活动记录 (Activation Records / Frame)



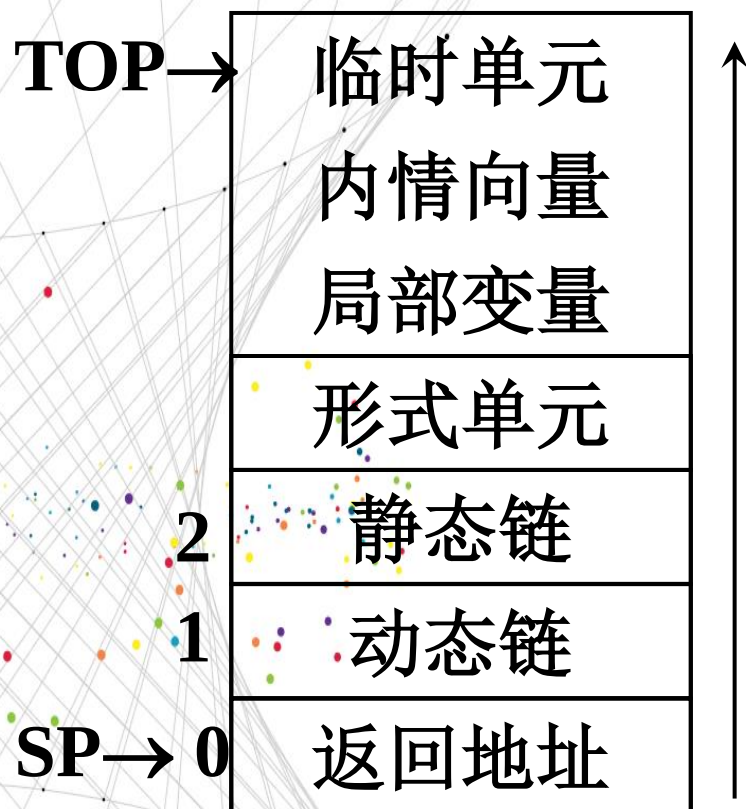
➤ **返回地址**

➤ **动态链**：指向调用该过程前的最新活动记录地址的指针。

➤ **静态链**：指向静态直接外层最新活动记录地址的指针，用来访问非局部数据。

运行时，每当进入一个过程就有一个相应的活动记录累筑于栈顶。

活动记录 (Activation Records / Frame)



- **形式单元**: 存放相应的实在参数的地址或值。
- **局部数据区**: 局部变量、内情向量、临时工作单元（如存放对表达式求值的结果）。

运行时，每当进入一个过程就有一个相应的活动记录累筑于栈顶。

活动记录 (Activation Records / Frame)

```
program main;
```

```
  procedure p;
```

```
    var a:real;
```

```
    procedure q;
```

```
      var b:integer;
```

```
      begin ... end;
```

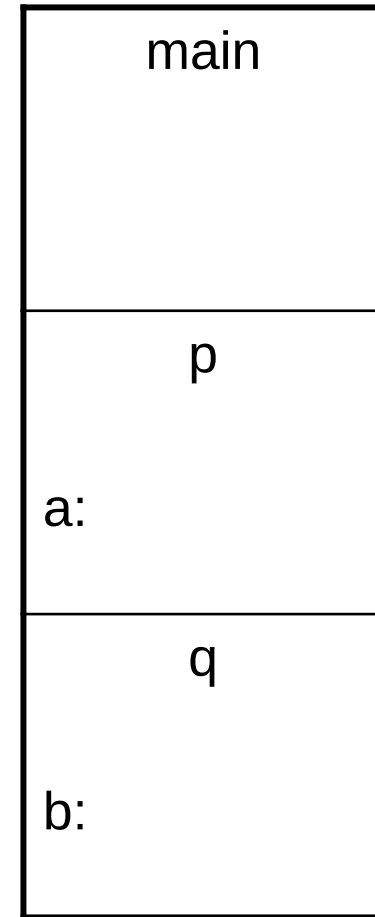
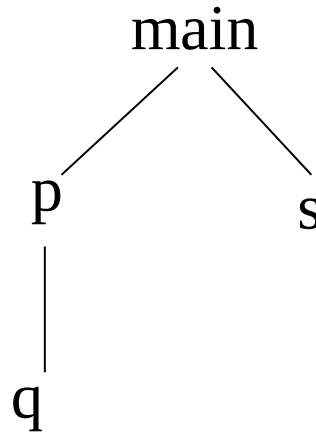
```
    begin q; end;
```

```
  procedure s;
```

```
    var c:integer;
```

```
    begin ... end;
```

```
  begin p; s; end;
```



活动记录 (Activation Records / Frame)

program main;

procedure p;

function q(a:integer):integer;

begin

if (a=1) then q:=1;

else q:=a+q(a-1);

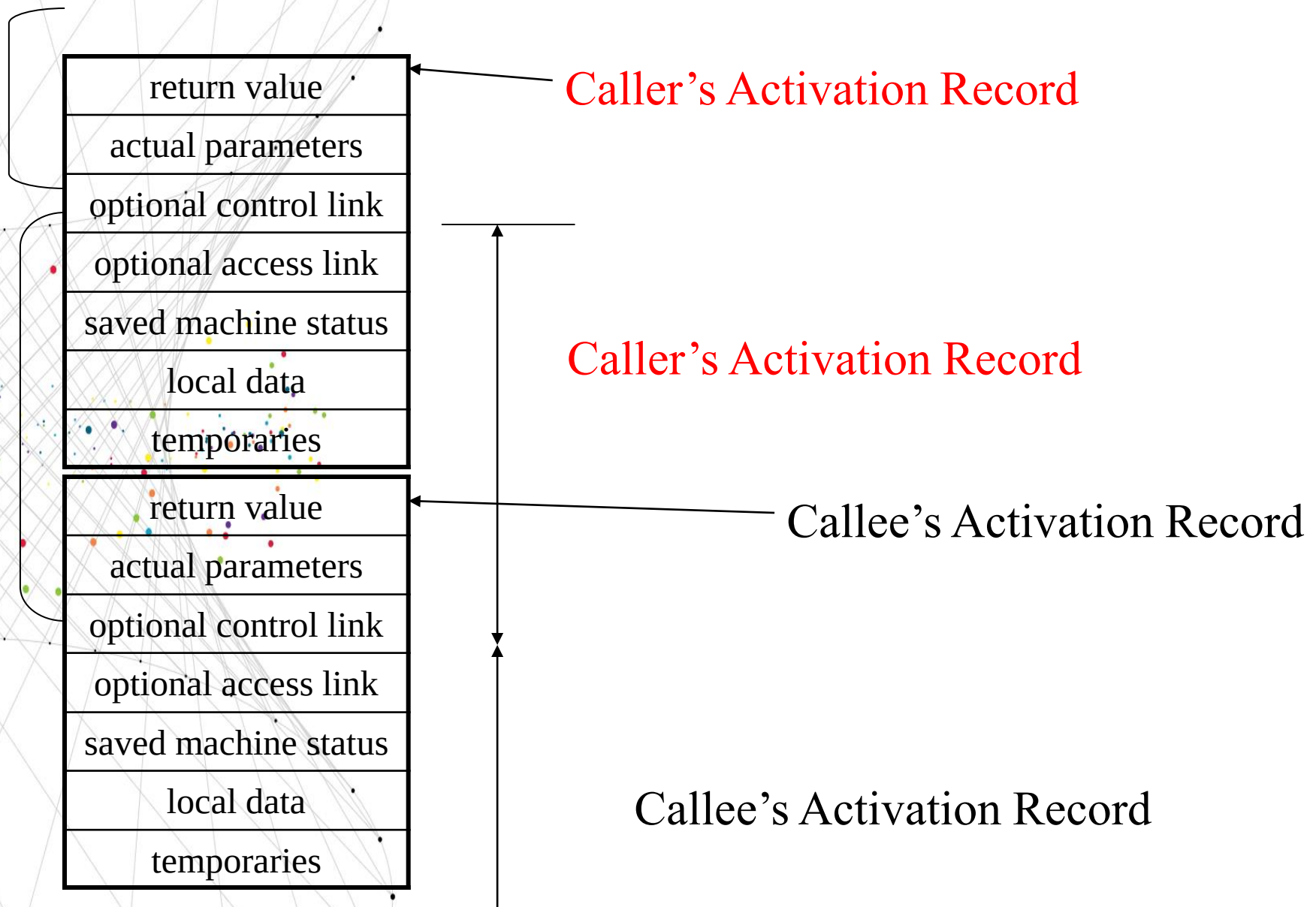
end;

begin q(3); end;

begin p; end;

main	
p	
a: 3	q(3)
a:2	q(2)
a:1	q(1)

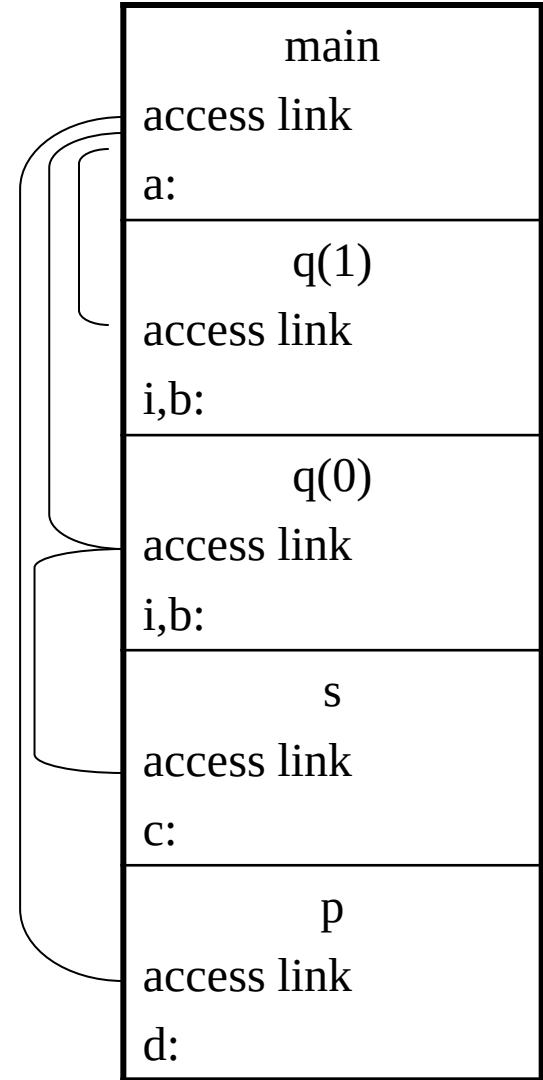
活动记录 (Activation Records / Frame)



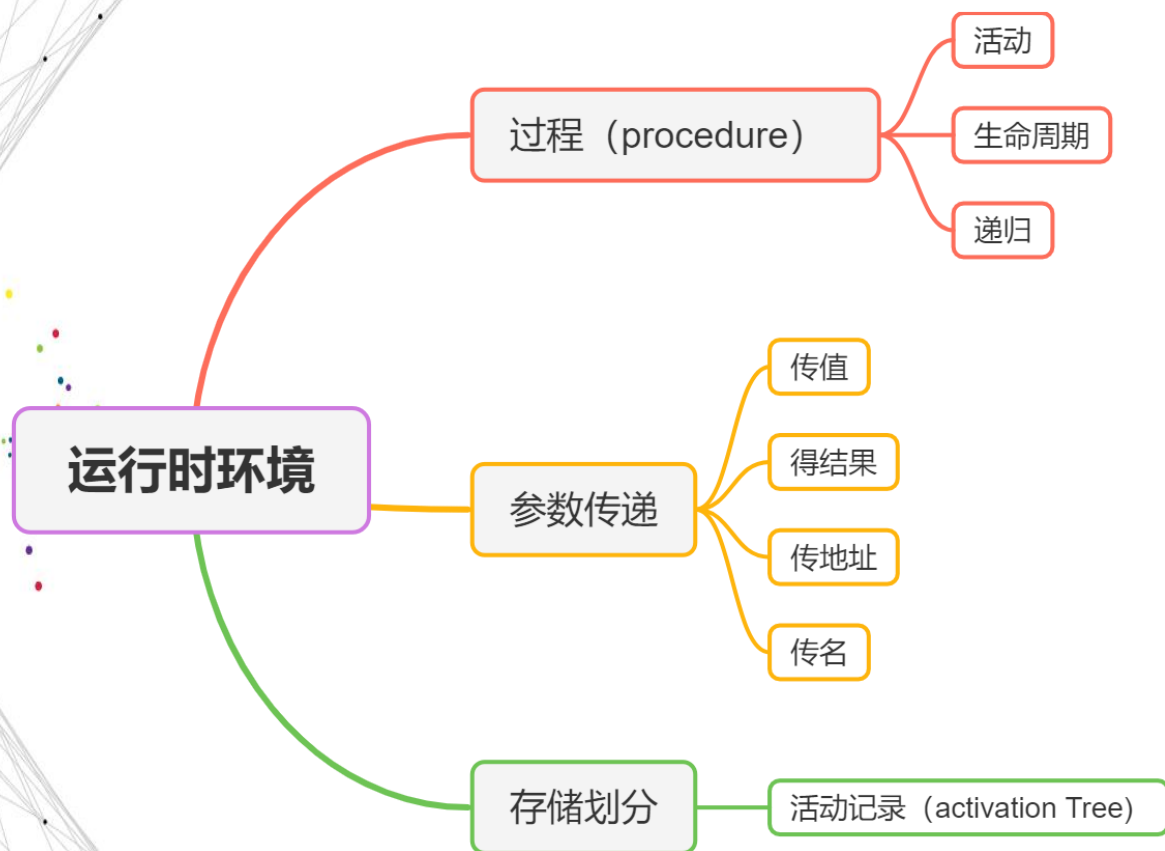
链路 (Access Links)

```
program main;  
  var a:int;  
  procedure p;  
    var d:int;  
    begin a:=1; end;  
  procedure q(i:int);  
    var b:int;  
    procedure s;  
      var c:int;  
      begin p; end;  
    begin  
      if (i<>0) then q(i-1)  
      else s;  
    end;  
  begin q(1); end;
```

Access
Links



核心概念总结



面向对象语言的编译器中运行时环境如何设计？